



Research Summary

Memory forensics: The path forward

Andrew Case ^a , Golden G. Richard III ^b  

Show more 

 Share  Cite

<https://doi.org/10.1016/j.diin.2016.12.004> 

[Get rights and content](#) 

Abstract

Traditionally, digital forensics focused on artifacts located on the storage devices of computer systems, mobile phones, digital cameras, and other electronic devices. In the past decade, however, researchers have created a number of powerful memory forensics tools that expand the scope of digital forensics to include the examination of volatile memory as well. While memory forensic techniques have evolved from simple string searches to deep, structured analysis of application and kernel data structures for a number of platforms and operating systems, much research remains to be done. This paper surveys the state-of-the-art in memory forensics, provide critical analysis of current-generation techniques, describe important changes in operating systems design that impact memory forensics, and sketches important areas for further research.

Introduction

Traditional storage forensics comprises a set of techniques to recover, preserve, and examines digital evidence and has applications in a number of important areas, including investigation of child exploitation, identity theft, counter-terrorism, intellectual property disputes, and more. Storage forensics tools are focused primarily on “dead” analysis, typically using bit-perfect copies of storage media. From these copies, deleted files or file fragments are recovered, patterns of file access are determined, past web browsing activity is observed, etc. Over the past decade, a number of factors have contributed to an increasing interest in *memory forensics* techniques, which allow analysis of a system's volatile memory for forensic artifacts. These factors include a huge increase in

the size of forensic targets, larger case back-logs as more criminal activity involves the use of computer systems, the use of forensics techniques in incident response to combat malware, and trends in malware development, where malware now routinely leaves no traces on non-volatile storage devices. Importantly, memory forensics techniques can reveal a substantial amount of volatile evidence that would be completely lost if traditional “pull the plug” forensic procedures were followed. This evidence includes lists of running processes, network connections, fragments of volatile data such as chat messages, and keying material for drive encryption.

While there has been tremendous progress in building advanced memory forensics tools since the first rudimentary techniques were developed around 2004, much work remains to be done in this exciting research area. This paper surveys the state-of-the-art in a number of areas in memory forensics, including acquisition and analysis, and attempts to clearly lay out the research challenges that lie ahead. These challenges include not only work that remains to be done, such as better analysis techniques for user-level malware, but also fundamental shifts in how memory forensics tools are designed and how they operate, to accommodate significant changes in operating systems design.

Access through your organization

Check access to the full text by signing in through your organization.

Access through **your organization**

Section snippets

Historical approaches to memory acquisition

The ability to acquire volatile memory in a stable manner is the first prerequisite of memory analysis. Traditionally, memory acquisition was a straightforward process as operating systems provided built-in facilities for this purpose. These facilities, such as `/dev/mem` on Linux and Mac OS X and `\\.\Device\PhysicalMemory` on Windows, provided administrator-level users direct access to physical memory.

On modern systems, such facilities are generally not available, however, due to security ...

Historical approaches to memory analysis

Memory analysis started in the early 2000s when digital forensic investigators realized that they could acquire memory directly from a running system through previously available interfaces. At

the time, there were both rootkits that were difficult or impossible to detect on a running system as well as anti-forensics techniques that could fool live analysis tools. At this stage in the history of memory forensics, however, there were no mature frameworks for performing memory analysis, and ...

Technologies without memory forensics coverage

So far, we have discussed platforms and operating systems for which memory forensics capabilities exist, but which require further work. We conclude the paper by discussing other important systems for which memory forensics is sorely needed, but for which no physical memory analysis capabilities exist. ...

Closing thoughts

Memory forensics has proven to be one of the most versatile and powerful ways to analyze computer systems. It has become a daily part of incident response procedures as well as the driving force behind proactive analysis of environments for malicious activity. In this paper we discussed historical work that was performed to enable the current power of memory forensics as well as improvements that should be made in order to ensure that memory forensics stays at the forefront of defensive ...

[Special issue articles](#) [Recommended articles](#)

References (57)

N. Petroni *et al.*

[Fatkit: a framework for the extraction and analysis of digital forensic data from volatile system memory](#)

Digit. Investig. (2006)

B. Schatz

[Bodysnatcher: towards reliable volatile memory acquisition by software](#)

Digit. Investig. (2007)

J. Stüttgen *et al.*

[Robust linux memory acquisition with minimal target impact](#)

Digit. Investig. (2014)

J. Sylve *et al.*

Acquisition and analysis of volatile memory from android devices

Digit. Investig. (2012)

J. Sylve *et al.*

Modern windows hibernation file analysis

Digit. Investig. (2017)

A.I. Ali-Gombe

Volatile Memory Message Carving: a “Per Process Basis” Approach

(2012)

Amazon EC2 Container Service Developer Guide(2016)

Android

Art and Dalvik

(2016)

Apple

Swift Developer Documentation

(2016)

M. Burdach

Physical Memory Forensics

(2006)



[View more references](#)

Cited by (0)

[View full text](#)

© 2017 Elsevier Ltd. All rights reserved.

